



When the Machine Has to Keep Going

Looping, Search, and the Shape of Repetition

Worksheet 02 • Estimated time: 4–5 hours • Solo build sprint

What this worksheet is not. It is not a tutorial. It is not a set of exercises. Nothing here has been pre-digested for you. You will attempt things before you understand them. That discomfort is structural, not accidental. Work with it.

What you will leave with. A working mental model of looping: repetition, stopping conditions, and search, calibrated through your own failures, rebuilt into something you actually understand.

How to use this. Read the problem. Attempt it immediately. Do not Google yet. Write down exactly where you get stuck. The obstacle is the path.

Name

Date

Branch

if this then that

Loop

do this until done

Jump

go handle that and come back

↓
Today's focus

1 Mission Brief

MISSION BRIEF

Some programs make one decision and stop. Useful programs rarely get that luxury.

A delivery app keeps searching for a route until it finds one that works. A payment system keeps checking transactions until the queue is empty. A simulation keeps advancing time until the model stabilises or explodes. A tiny script keeps testing numbers until a pattern shows itself.

This worksheet is about looping. Not the syntax. The *discipline*.

Today you will build programs that repeat work without getting lost. You will generate number patterns, hunt for primes, pull digits out of integers, convert between number systems, and search through possible triangles. The hard part is not writing `for` or `while`. The hard part is knowing what changes each time, what must stay remembered, and exactly when the machine should stop.

Most broken loops do not look broken at first. They run one time too many, one time too few, or forever. Your job is to find those edges before they find you.

2 Today's Focus — Looping: “Do This Until Done”

A loop is controlled repetition. It is what you use when a program needs to:

- repeat the same kind of work,
- process a sequence one item at a time,
- continue until a condition becomes false,
- generate values repeatedly,
- count, accumulate, or search through possibilities.

Before code, think in ordinary language:

- Keep walking until you reach the destination.
- Keep adding bills until the total exceeds the budget.
- Keep checking numbers until a pattern appears.
- Repeat the process until there is nothing left to process.

Core skill you are training: Looking at repeated work and identifying the changing value, the memory you carry forward, and the stopping condition. This is not about Python. It is about control.

By the end of this worksheet you will have:

- A set of working Python programs built around loops.

- A clean distinction between “first N values” and “all values up to a limit”.
 - A debugging log for off-by-one errors, infinite loops, and broken boundary cases.
 - A one-page technical note explaining how loops generate, search, and accumulate.
-

3 Build Before You Know — Pattern Hunts

Do not read ahead. Run tiny experiments first. Your job is to notice what changes, what repeats, and where the process should stop.

Experiment A

Keep Doubling

TRY IT NOW

Start with $x = 1$. Keep doubling it. Print every value until it becomes greater than 1000.

Before coding: How many values do you think will appear?

What changed each time? What stopped the loop?

Experiment B

The Two-Number Machine

TRY IT NOW

Start with two numbers: 0 and 1. Keep making the next number by adding the previous two.

Print the first 10 values. What pattern emerges?

Which two values did your program need to remember?

Experiment C

Keep Cutting in Half

TRY IT NOW

Start with $x = 100$. Keep dividing by 2. Stop when x becomes smaller than 1.

Question: Does the loop stop at exactly 1, below 1, or after passing below 1?

Write the exact stopping rule in plain English:

Experiment D

Check Numbers One by One

TRY IT NOW

Check every number from 1 to 30. Print the number if it is divisible by 3 but not by 2.

Do not optimise. First make the loop honest.

What part repeated? What condition filtered the results?

4 What You Are Building — Just Enough Theory

JUST-IN-TIME

Two loop shapes.

```

1  # for loop: useful when you know the sequence you want to walk
       through
2  for i in range(10):
3      print(i)
4
5  # while loop: useful when you know the stopping condition
6  x = 1
7  while x <= 1000:
8      print(x)
9      x = x * 2

```

A loop needs three pieces of architecture:

- **initialisation:** where the repeated process begins,
- **update:** what changes after each pass,
- **termination condition:** when the machine stops.

JUST-IN-TIME

Counters and accumulation.

counter	a value that tracks how many times something has happened
accumulator	a value that carries a result forward, like a sum or product
iteration	one pass through the loop
stopping condition	the rule that prevents the loop from running forever

```

1  total = 0
2  for n in range(1, 6):
3      total = total + n
4  print(total)  # 15

```

JUST-IN-TIME

Fibonacci Variant A: first N terms.

Generate the first 10 Fibonacci numbers:

```

1  a = 0
2  b = 1
3
4  for count in range(10):
5      print(a)
6      next_value = a + b
7      a = b
8      b = next_value

```

This loop stops because the **counter** reaches 10. It does not care how large the numbers become.

JUST-IN-TIME**Fibonacci Variant B: terms below a limit.**

Generate all Fibonacci numbers less than 1000:

```
1 a = 0
2 b = 1
3
4 while a < 1000:
5     print(a)
6     next_value = a + b
7     a = b
8     b = next_value
```

This loop stops because the **next printed value** crosses a numerical boundary. It might print 5 values, 16 values, or 100 values depending on the limit.

Do not blur these two ideas: “first N Fibonacci numbers” means the loop is controlled by a count. “Fibonacci numbers below 1000” means the loop is controlled by a boundary.

Transition. Fibonacci taught us how to generate repeated values. Prime numbers ask a different question: how do we keep checking numbers until we find valid ones?

5 Looping Field Problems

For each problem, build first. Then attack the collision points. You are not trying to avoid mistakes. You are trying to make them visible.

Problem 1

Prime Check

BUILD NOW

Write a program that takes an integer and prints whether it is prime.

Think about: divisibility, trial division, and whether you can stop early.

Test it on: 1, 2, 3, 4, 17, 25, 97, -7

Your first rule for prime:

Collision points to mark: Is 1 prime? Is 2 prime? What should happen for negative numbers?

Problem 2

Generate All Primes Up to N **BUILD NOW**

Given a number N , print every prime number $\leq N$.

Example: if $N = 20$, the output should be 2, 3, 5, 7, 11, 13, 17, 19.

Where does the outer loop run? Where does the prime check happen?

Collision points to mark: $N < 2$, repeated checks, slow code for large N .

Problem 3

Generate the First N Primes**BUILD NOW**

Given a count N , print the first N prime numbers.

Example: if $N = 20$, print the first 20 primes, even though the final prime is greater than 20.

Write the difference in plain English: primes up to N vs first N primes.

Collision points to mark: $N = 0$, stopping after exactly N primes, not after checking numbers up to N .

Problem 4

Sum of Digits

BUILD NOW

Take an integer and compute the sum of its digits.

Example: $4832 \rightarrow 4 + 8 + 3 + 2 = 17$

Constraint: Do it once using arithmetic: repeated $\%$ 10 and $//$ 10. Then compare with a string-based version.

What value shrinks each time? What value accumulates?

Collision points to mark: input 0, negative numbers, trailing zeroes like 1200.

Problem 5

Factorial

BUILD NOW

Compute $n!$.

Example: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Focus: accumulation through multiplication.

What should the accumulator start as, and why?

Collision points to mark: $0!$, negative input, very large n .

Problem 6

Decimal to Binary

BUILD NOW

Convert a decimal integer into binary manually using loops.

Hint: Repeatedly divide by 2. The remainders tell the story, but not in the order you first see them.

Test it on: 0, 1, 2, 5, 13, 32

Where do the remainders go? Why might the answer come out backwards?

Collision points to mark: decimal 0, powers of 2, reversing the result.

Problem 7

Binary to Decimal

BUILD NOW

Convert a binary number into decimal.

Example: $1011_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$

Focus: positional reasoning. Each digit has a place value.

Will you scan left-to-right or right-to-left? What changes each step?

Collision points to mark: invalid digits, leading zeroes, input stored as an integer vs a string.

Problem 8

GCD and LCM

BUILD NOW

Given two integers, compute their GCD and LCM.

Version 1: Try iterative divisor checking.

Version 2: Try the Euclidean algorithm.

Then: derive LCM from GCD.

Which loop feels more elegant? Which loop does less work?

Collision points to mark: one number is 0, both numbers are equal, negative inputs.

Problem 9

Pythagorean Triplets Below N **FINAL CHALLENGE**

Given N , generate all triplets (a, b, c) below N such that:

$$a^2 + b^2 = c^2$$

No need to deduplicate scaled versions. Both $(3, 4, 5)$ and $(6, 8, 10)$ are acceptable.

Goal: make the loop bounds clear enough that your program does not waste absurd work or miss valid triplets.

How many loops do you think you need? Can one of the bounds be tighter?

Collision points to mark: duplicate ordering, loop bounds, whether $a < b < c$ should be enforced.

6 Collision Points — Where Loops Lie

These are the breakpoints most people hit. Check each one against your code.

COLLISION POINT**Collision 1: One too many, one too few.**

If your loop says `range(1, N)`, it stops before N . If your condition says `while x < limit`, it behaves differently from `while x <= limit`. This is not a tiny syntax detail. It decides whether boundary values are included.

COLLISION POINT**Collision 2: Fibonacci count vs Fibonacci boundary.**

First N Fibonacci numbers and Fibonacci numbers below a limit are different programs. One stops because a counter reaches N . The other stops because the generated value crosses a boundary.

Test: What should happen when $N = 0$? What should happen when the limit is smaller than 1?

COLLISION POINT**Collision 3: The prime number 2.**

Many prime checks accidentally reject 2 because they treat even numbers as not prime. That shortcut is only valid after 2 has been handled. Also: 1 is not prime, and negative numbers are not prime.

COLLISION POINT**Collision 4: Searching forever for the first N primes.**

When generating the first N primes, you do not know the final number in advance. Your stopping condition cannot be “candidate reaches N ”. It has to be “count of primes found reaches N ”.

COLLISION POINT**Collision 5: Accumulators need the right identity.**

For summing digits, the accumulator starts at 0 because adding 0 changes nothing. For factorial, the accumulator starts at 1 because multiplying by 1 changes nothing. Start factorial at 0 and every answer becomes 0.

COLLISION POINT**Collision 6: Decimal to binary reverses the evidence.**

Repeated division by 2 gives remainders from least significant bit to most significant bit. If you append them normally, your answer is backwards. Handle decimal 0 separately before the loop.

COLLISION POINT**Collision 7: GCD with zero.**

`gcd(a, 0)` should be `abs(a)`. But `lcm(a, 0)` is 0. If you compute LCM as `abs(a*b)//gcd(a,b)` without thinking, the GCD part is fine, but the meaning still deserves a deliberate branch.

COLLISION POINT**Collision 8: Triplets can repeat in disguise.**

If you loop over every a , b , and c , you may print both $(3, 4, 5)$ and $(4, 3, 5)$. Decide whether ordering matters. For this worksheet, scaled versions like $(6, 8, 10)$ are allowed. Swapped duplicates are just noise.

Go back to your programs. Mark every broken assumption before you fix it.

7 Just-in-Time Knowledge**JUST-IN-TIME****The loop checklist.**

Before trusting any loop, answer these five questions:

- What is the first value?
- What changes after one iteration?
- What is being remembered across iterations?
- What exact condition stops the loop?
- What are the smallest and strangest inputs?

JUST-IN-TIME**Loops have personalities.**

Generate	keep producing values: Fibonacci, powers of 2, binary digits
Search	keep checking candidates: primes, triplets, divisors
Accumulate	keep carrying a result: digit sum, factorial, total cost
Reduce	keep shrinking the problem: Euclidean GCD, decimal conversion

JUST-IN-TIME**A good loop fails at the boundaries first.**

Test these before you trust the happy path:

- zero values,
- one-item inputs,
- values exactly at the limit,
- values just below and just above the limit,

- negative numbers when the problem claims to use integers.

SHIP IT**Ship it.**

Choose any three programs from this worksheet and rebuild them cleanly. For each one, include:

- the final Python code,
- five test cases, including at least two collision points,
- a short note explaining the loop architecture: initialisation, update, memory, stopping condition.

Then write one page titled: *What loops taught me about control*. Do not summarise syntax. Explain one assumption that broke and how you repaired it.

TASTE CALIBRATION**Taste calibration.**

A loop that works is not always a loop you should trust. Compare two of your solutions and decide which one is clearer. Look for:

- fewer hidden assumptions,
- cleaner stopping conditions,
- less duplicated logic,
- test cases that make failure obvious.

Elegance here is not cleverness. It is a program whose stopping point is impossible to misunderstand.